Ollscoil na Gaillimhe
University of Galway

Improving Conversational Recommendation Systems with Large Language Models

Cathal Lawlor
Supervisor: Dr. Colm O'Riordan

School of Computer Science, University of Galway, Ireland

April 7, 2025

Contents

Chapter 1

Introduction

1.1 Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Colm O'Riordan, for his guidance and support throughout this project and for allowing me to work on a project that I truly enjoyed and that provided me with an invaluable learning experience.
I am also grateful to Pearse Carroll and Joe O'Connell for their assistance in providing access to the necessary resources. Additionally, I would like to extend my thanks to Gergely Toth for his valuable advice and insights regarding the implementation of large language models in this project.

1.2 Glossary

- **LLM:** Large Language Model
- **CRS:** Conversational Recommendation System
- **IR:** Information Retrieval
- **NLP:** Natural Language Processing
- **CF:** Collaborative Filtering
- **CBF:** Content-Based Filtering
- **FYP:** Final Year Project
- **ML:** Machine Learning
- **UX:** User Experience

1.3 Domain Background

The background to this project is based on the desire for a more personalised and transparent recommendation system that allows users to provide feedback on the recommendations made to them in a conversational manner. Existing recommendation systems have limitations in terms of personalisation and transparency, with users often feeling frustrated and powerless when they want to change, correct, or adjust the recommendations made to them.

Current recommendation systems often rely on collaborative filtering and content-based filtering techniques, which can lead to a lack of personalisation and transparency in the recommendations made to users. These systems are often opaque, with users having little understanding of how the recommendations are made, leading to a lack of trust in the system. Even when users are provided with explanations for the recommendations made to them, these explanations are often not user-friendly and can be difficult to understand.

Collaborative filtering techniques rely on the behaviour of other users to make recommendations, by

using algorithms to find similar users and recommend items that those users have liked. Content-based filtering techniques rely on the attributes of the items being recommended, by using algorithms to find similar items and recommend those items to users. These techniques can lead to a lack of personalisation, particularly when user preferences are not well captured.

Additionally, while large language models (LLMs) have shown promise in the area of conversational recommendation systems, they often do not have the capability to handle large context windows for the size of data needed when the system is used outside of a single session. Furthermore, LLMs often do not have the capability to store fine-grain memory of conversations, losing information on implicit user preferences and feedback. Alongside this, the lack of reasoning capabilities in LLMs can lead to a lack of transparency in the recommendations made to users, as the explanations are at best, guessed by the model. New research in the area of reasoning models may help to alleviate this issue[24].

1.4 Problem Statement

The project aims to address the following key problems:

- Traditional recommendation systems have limitations in terms of personalisation and transparency.
- Users often feel frustrated and powerless when they want to change, correct, or adjust the recommendations made to them, leading to a lack of trust in the system.
- Existing recommendation systems do not allow for user feedback other than a rating, limiting user control over the recommendations.
- Large language models (LLMs) have shown promise in conversational recommendation systems but face challenges such as:
  - Limited context windows for handling large datasets without requiring massive models.
  - Long term memory issues, where the model may not remember user preferences and feedback across sessions.
  - Lack of fine-grain memory of conversations, resulting in the loss of specific user preferences and feedback.
  - Limited reasoning capabilities, leading to a lack of transparency in the recommendations and explanations provided.
- Users want to feel heard and desire a system that is transparent, user-friendly, and capable of providing personalised recommendations.
- There is a need to explore how conversational recommendation systems can store user profiles (e.g., films recommended, watched, etc.) to enhance personalisation and transparency.

1.5 Aims and Objectives

The primary aim of this project is to explore and investigate the integration of large language models (LLMs) with recommendation systems to enhance personalisation, transparency, and user control. The project seeks to address the limitations of traditional recommendation systems by leveraging conversational interfaces and advanced AI techniques. The specific objectives of the project are as follows:

- To explore the potential of conversational recommendation systems in improving user experience and engagement.
- To investigate the use of large language models for generating personalised and transparent recommendations.
- To design and implement a user-adjustable recommendation system that allows users to provide feedback and adjust recommendations in a conversational manner.
- To evaluate the effectiveness of LLMs in providing clear and user-friendly explanations for recommendations.

- To assess whether the proposed system can increase user trust, control, and satisfaction with the recommendations.
- To utilise the capabilities of LLMs to analyse user feedback and adjust recommendations based on user preferences.
- To explore the use of LLMs in generating conversation to enhance user engagement and interaction with the recommendation system.
- To explore the role of conversational feedback in addressing the cold start problem in recommendation systems.
- To investigate the integration of LLMs with traditional information retrieval systems for enhanced recommendation capabilities.
- To evaluate the performance of low-intensity LLMs, such as quantized and instruction-based models, in handling multiple tasks within a recommendation system context.

By achieving these objectives, the project aims to contribute to the development of more user-centric and transparent recommendation systems, advancing the state of the art in the field of conversational AI and recommendation technologies.

1.6 Open Questions

The project aims to address the following open questions:

1. **Enhancing Conversational Recommendation Systems:** How can large language models improve the effectiveness of conversational recommendation systems?
2. **User Feedback Mechanisms:** How can users provide feedback on recommendations through a conversational interface?
3. **User-Adjustable Systems:** What approaches can be used to design intuitive and accessible user-adjustable recommendation systems?
4. **Explainability:** In what ways can large language models deliver clear and user-friendly explanations for recommendations?
5. **Accessibility:** How can we develop a system that is easy to use for individuals without a technical background?
6. **Scalability and Efficiency:** What strategies can ensure the system is scalable, efficient, effective, and user-centric?

1.7 Contributions

The contributions of this project are as follows:

- **Novel Integration of LLMs with Recommendation Systems:** This project explores the integration of large language models with traditional recommendation systems to enhance personalisation and transparency.
- **Development of a Conversational Interface:** A conversational interface is implemented to allow users to interact with the recommendation system, providing feedback and adjusting recommendations in a user-friendly manner.
- **Explainability in Recommendations:** The project investigates the use of LLMs to generate clear and comprehensible explanations for recommendations, improving user trust and understanding.
- **User-Adjustable Recommendation System:** A proof-of-concept system is developed to demonstrate how users can adjust recommendations based on their preferences through conversational feedback.

- **Addressing the Cold Start Problem:** The project explores how conversational feedback and user profiles can alleviate the cold start problem in recommendation systems.
- **Evaluation of Low-Intensity LLMs:** The project evaluates the capabilities of quantized and instruction-based LLMs in handling multiple tasks within a recommendation system context.

1.8 Report Structure

The report will be structured as follows:

Chapter One provides an introduction to the project, outlining the aims and objectives of the project, the problem statement and domain background, and the structure of the report.

Chapter Two provides a review of the literature in the area of conversational recommendation systems, large language models, and user feedback in recommendation systems.

Chapter Three outlines the design of the system, including the system architecture, the design decisions made, and the data used in the project.

Chapter Four details the implementation of the system, including the technologies and languages used, and the challenges faced during implementation.

Chapter Five presents the evaluation criteria of the system. It includes a discussion of the classic metrics used in recommendation systems, and the user evaluation process.

Chapter Six presents the results and evaluation of the system, including sample run-throughs with sample queries and recommendations, and the evaluation of the system.

Chapter Seven provides a conclusion to the project, summarising the main findings, reflecting on the project, and discussing future work.

Appendices include additional information on the project, such as code snippets, data samples, and user feedback forms.

Chapter 2

Related Work

Conversational recommendation systems (CRSs) represent a rapidly evolving intersection of artificial intelligence, natural language processing, and user interaction design, aimed at enhancing personalized experiences across diverse domains such as e-commerce, entertainment, and customer service. By facilitating engaging dialogues between users and machines, CRSs provide dynamic and context-aware recommendations tailored to individual preferences, effectively addressing the complexity of choices available in today's digital landscape.

2.1 Large Language Models as recommendation systems

This study from Ruixuan Sun et al [22] looked at using large language models as a recommendation system. This study found that LLMs offer strong recommendation explainability but lack overall personalization, diversity, and user trust.

2.1.1 Leveraging Large Language Models in Conversational Recommender Systems

Recent developments[2] in large language models (LLMs) have significantly improved the capabilities of conversational recommenders. Notably, studies have demonstrated that LLMs can serve as effective zero-shot conversational recommenders, allowing systems to generate recommendations based on limited prior knowledge of user preferences. These advancements are complemented by the integration of tools and external knowledge bases, enabling systems to perform complex tasks and provide more accurate recommendations

2.1.2 Collaborative Retrieval for LLM-based CRSs

This paper by Zhu et al. [24] introduces CRAG (Collaborative Retrieval Augmented Generation), a novel approach that combines large language models (LLMs) with collaborative filtering (CF) for conversational recommender systems (CRSs). While LLMs excel at understanding context and user preferences in conversations, they often lack the ability to leverage behavioural data effectively, which is a strength of traditional collaborative filtering methods. CRAG addresses this limitation by integrating both approaches.

The CRAG framework consists of three main components:

- **LLM-based entity linking:** Identifies and links entities mentioned in conversations.
- **Collaborative retrieval with context-aware reflection:** Utilizes collaborative filtering to retrieve relevant items based on historical interaction data.
- **Recommendation with reflect-and-rerank methodology:** Ranks recommendations by combining insights from both LLMs and collaborative filtering.

The authors demonstrate CRAG's superior performance on two movie recommendation datasets, namely a refined Reddit dataset (Reddit-v2) and the Redial dataset. The results show that CRAG provides better item coverage and recommendation accuracy compared to baseline CRS methods, with particularly significant improvements for recently released movies.

By integrating collaborative filtering with LLMs, CRAG bridges a critical gap in current CRS approaches. This integration enables the system to leverage behavioural data that is not typically included in LLM training, resulting in improved recommendation performance. Additionally, the

refined Reddit dataset introduced in this study highlights the importance of high-quality data in advancing research in this field.

2.2 Critiquing-based recommendation systems

The project touches in on critiquing-based recommendation systems, where the user is asked to provide feedback on what preferences they hold for recommendations, with the system using this feedback to refine the user profile and improve the recommendations given to the user. Current recommendation systems are limited in that they are only based on static attribute values (for example, in a digital marketplace, attributes such as a digital camera's screen size, effectiveness pixels, optical zoom).

One paper, Chen et al[9], proposes a sentiment-integrated critiquing approach, for helping users to formulate and refine their preferences, improving the user's product knowledge, preference certainty, decision confidence, perceived information usefulness, and purchase intention.

This can be integrated into a conversational recommender system, where the user sentiment can be used to refine the recommendations given to the user (e.g. the system might is picking up that a user isn't interested in a particular genre of movie, but isn't explicitly saying so, so the system can ask the user for feedback on the genre of movie they are being recommended).

2.3 Data scarcity and training cost

Data scarcity remains a significant challenge in training effective Conversational Recommender Systems (CRSs). To address this issue, techniques such as model distillation and fine-tuning can be employed to enhance model performance while reducing computational costs.

In the paper by Hsieh et al. [16], the authors propose a step-by-step distillation approach. This method involves training a smaller model to mimic the behaviour of a larger model, followed by fine-tuning the smaller model on the target task. This approach not only outperforms the larger model on the target task but also proves to be more computationally efficient.

Another strategy to mitigate data scarcity is the generation of synthetic data using a larger pre-trained model. This technique leverages the capabilities of a large language model to create additional high quality training examples, which can then be used to train a smaller, more efficient model. By augmenting the training dataset with synthetic data, the performance of the conversational recommender system can be improved without the need for extensive real-world data collection. This method is particularly useful in scenarios where obtaining large amounts of labelled data is challenging or costly. For instance, in this project, it is not feasible for to have a large number of users interacting with the system to generate training data.

2.4 Explainability

In conversational recommendation systems, explainability is crucial for building trust with users. The system must be able to clearly explain why a particular recommendation was made and how it arrived at that conclusion.

Current approaches to explainability in recommendation systems, such as those discussed by Guo et al. [14], or Xu et al. [23] focus on providing transparent and understandable justifications for recommendations. These approaches often involve breaking down the recommendation process into comprehensible steps that can be communicated to the user. For instance, a system might explain that a movie recommendation is based on the user's previous preferences for a specific genre or the popularity of the movie among similar users.

Recent advancements in natural language processing have enabled more sophisticated explainability mechanisms. NLP techniques can be used to generate natural language explanations that are easy for users to understand. Additionally, dialogue management systems can handle follow-up questions from users, providing further clarification and enhancing the overall transparency of the recommendation process.

This project aims to incorporate these explainability techniques to ensure that users feel confident and informed about the recommendations they receive. By leveraging both content-based and collaborative filtering explanations, as well as utilizing NLP for natural language justifications, we can create a conversational recommendation system that not only provides accurate recommendations but also builds trust and engagement with users.

2.4.1 Context-aware recommendation

While not directly applicable to the project, this paper from Carchiolo et al[5] proposes a context-aware recommendation network, which can be used to find experts in a particular field.

If this was applied to a conversational recommender system, it could be used to find users with a lot of experience in a particular field, and use their feedback to improve the recommendations given to other users. This approach builds upon traditional information retrieval systems, which are already designed to address challenges posed by low-quality users, such as those who have contributed only one or two reviews in total.

2.5 Current Evaluation Methods

2.5.1 Evaluation of Recommender Systems

Traditional evaluation methods for accuracy and evectiveness of recommender systems are based on classic metrics, such as precision, recall, and F1 score. These metrics are used to evaluate the accuracy of the recommendations made by the system.

However, these metrics have limitations when it comes to capture and evaluate the effectiveness of the user experience with conversational recommender systems, with nuanced interactions and user satisfaction.

2.5.2 Evaluation of Conversational Recommender Systems

The evaluation of conversational recommender systems is a multifaceted challenge that encompasses various dimensions such as recommendation accuracy, user satisfaction, and interaction quality. Traditional metrics for recommender systems, as mentioned above in Section 2.5.1, such as precision, recall, and F1-score are useful for measuring the accuracy of recommendations.
However, these metrics alone are insufficient to capture the interactive and dynamic nature of CRSs.

Recent research has emphasized the importance of user-centric evaluation metrics, including user satisfaction, engagement, and perceived usefulness. User studies and A/B testing are commonly used to gather qualitative feedback and assess the overall user experience, but requires a large user base. Additionally, metrics such as dialogue coherence, response relevance, and interaction length are considered to evaluate the quality of the conversational interactions [21].

The evaluation of CRSs has also seen the emergence of novel approaches, such as the use of human-in-the-loop evaluations, where human annotators assess the quality of recommendations and interactions. This approach allows for a more nuanced understanding of user preferences and expectations [20]. Furthermore, the integration of user modelling techniques, such as user profiling and context-aware recommendations, has been explored to enhance the personalization and relevance of recommendations. These techniques aim to capture user preferences, behaviours, and contextual information to provide more tailored recommendations [19, 8].

The use of large language models (LLMs) in CRSs has introduced new challenges and opportunities for evaluation. LLMs can generate human-like responses, but their performance can vary significantly based on the training data and fine-tuning processes. While LLMs have their own evaluation metrics, such as perplexity and BLEU scores[6], these metrics do not fully capture the effectiveness of LLMs in the context of CRSs.
Moreover, the use of simulation environments and user simulators has gained traction[17] as a means to conduct large-scale evaluations without the need for extensive real-world user interactions. These simulators can model user behaviour and preferences, providing a controlled setting to test and refine CRSs. Despite these advancements, papers outline that there remains a need for standardized evaluation frameworks that can comprehensively assess the performance of CRSs across different domains and use cases.

Chapter 3

Design

3.1 System Architecture

The system architecture is designed to be modular, allowing for the addition of other systems or components. The architecture is composed of several interconnected components, each designed to handle specific functionalities within the system.

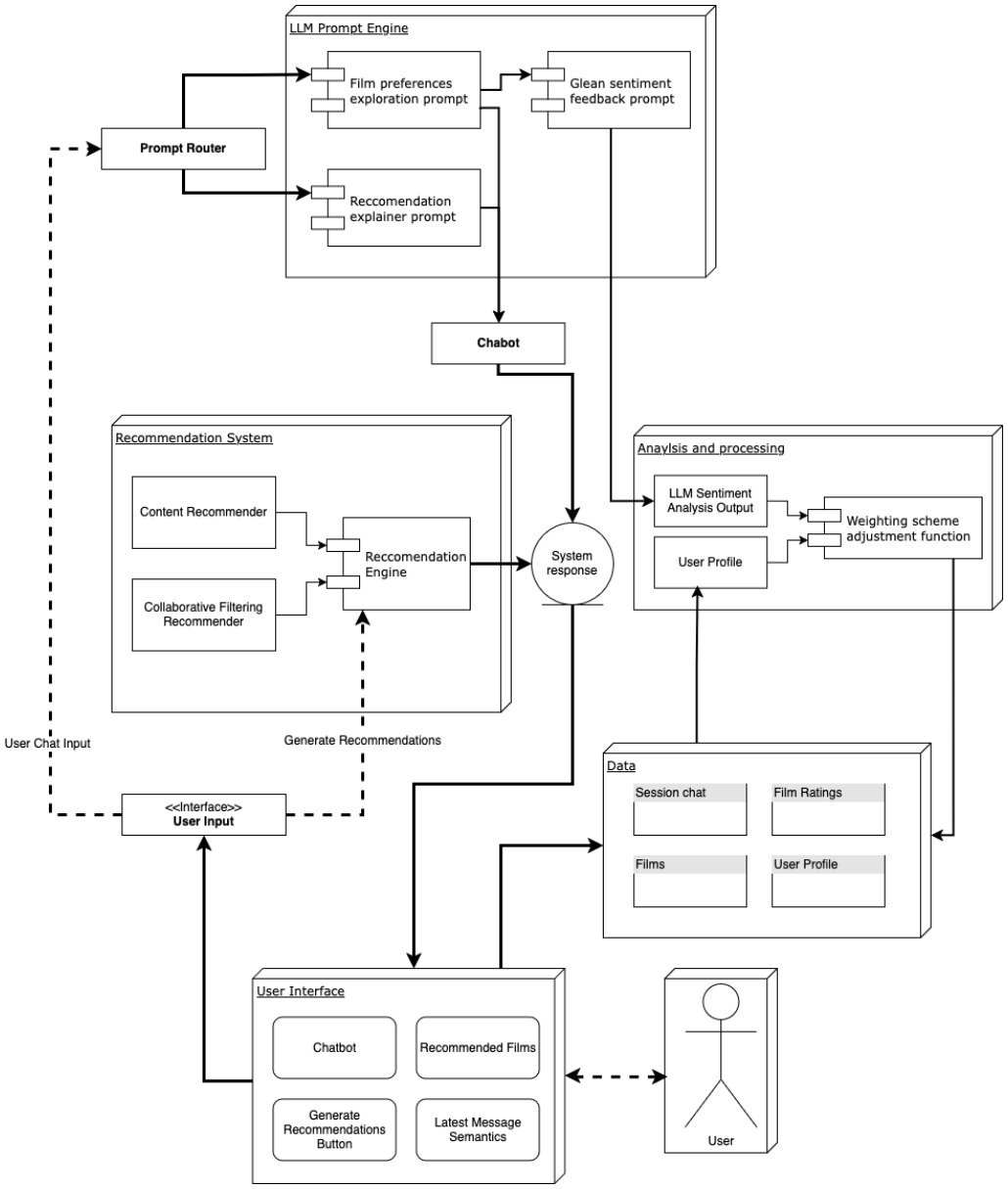

Figure 3.1: System Architecture Diagram

The system architecture diagram (Figure 3.1) shows the main components of the system, and how they interact with each other.

3.1.1 System Architecture Components

The system architecture adopts a modular design, enabling the integration and modification of new and existing components and features.

- **User Interface** - The user interface is designed to be simple and easy to use, allowing the user to interact with the system and provide feedback.
- **Recommendation System** - The recommendation system is designed to allow for the addition of new recommendation systems, features and the adjustment of existing features.
- **Data** - The data holds the session conversation logs, user preference profile, films and their features, and film ratings from users.
- **Data Analysis & Processing** - The data analysis and processing handles loading and preprocessing the data, processing sentiment analysis and adjustment of the user profile.
- **Large Language Model Prompt Engine** - The large language model prompt engine handles various scenarios, and routes the conversation to the appropriate component.

3.2 Hybrid Recommendation System

The recommendation system is designed to be both modular and adjustable. As there doesn't exist an existing framework, the recommendation system is designed to be a proof of concept, and is not intended to be the best implementation. It provides a working implementation of a recommendation system, on which the other components can be built.

The design choice to use a hybrid recommendation system is to provide a **more comprehensive recommendation system**, allowing for the use of both collaborative filtering and content-based filtering.

3.2.1 Content-based Filtering

As there's no frameworks or libraries for content-based filtering with the adjustability needed for recommendation, explanation and feature adjustment, a custom implementation is needed.

As mentioned above in Section 3.2, this implementation is to provide satisfactory recommendations based on the features of the films. The content-based filtering system uses the following features:

- Genres
- Directors
- Cast

Additional features could be added, such as the release year, but the listed features are sufficient for a substantial proof of concept that can be built upon and works well. Adding additional features would be a stretch goal, and would not be necessary for the proof of concept.

The content-based filtering system will fulfil the following requirements:

- The system is able to provide recommendations based on the features
- The system can provide recommendations based on the features with a user profile
- Feature weights can be adjusted to provide different recommendations
- The system provides a breakdown of the scores for each feature in the recommendation

With this in mind, the content-based filtering system will not be forensically examined, with the evaluation of the system being covered in Section 5.1.

3.2.2 Collaborative Filtering

The collaborative filtering component of the recommendation system leverages the LensKit library[11] to implement a user-user collaborative filtering approach. This method utilizes a user-item matrix to identify similar users and recommend items based on their preferences.

As it is a proven and widely used approach and the LensKit library is a well-established framework for collaborative filtering, it was chosen for its reliability and ease of integration into the overall system architecture. This component will not be evaluated in detail, as already mentioned above in Section 3.2, particularly as it is library that is well established and proven to work.

The collaborative filtering component will provide recommendations based on the user item matrix, and will be able to provide recommendations based on the current user profile.

3.3 Large Language Model Component

The system will utilise the LangChain framework [7] for the large language model component, as it provides a simple and easy to use interface for the large language model. This allows for the integration of the large language model with the other components of the system, and provides a simple interface for the user to interact with the system.

3.3.1 Prompt Routing

The integration will follow a finite state machine model, where the system will follow a set of states to determine the next action.

With this, the system will be able to handle the conversation and route the conversation to the appropriate component, based on the user's input. The system will follow a set of states to determine the next action, and will use the following states:

- **Greeting** - The system will greet the user and ask for their preferences.
- **Film Preference Exploration** - The system will converse with the user to explore their film preferences, asking questions about their favourite films, genres, directors, and actors.
- **Preference Extraction** - The system will extract the user's preferences from the conversation and store them in the user profile.
- **Recommendation Explanation** - When prompted, the system will explain the recommendations and provide a breakdown of the scores for each feature in the recommendation.

3.4 Data

3.4.1 Data Preprocessing

The system will include measures to handle cleaning the data by handling missing values, normalizing features, and encoding genres as they are categorical variables.

3.4.2 Data Storage

The system employs a dual-layered data persistence strategy, distinguishing between session data and user profile data. This design allows for greater flexibility and personalization in the recommendation process.

A user profile serves as a comprehensive repository of a user's preferences, while session data captures the context-specific preferences and interactions during a particular session. For instance, a user may maintain separate sessions for different contexts, such as one for family-friendly recommendations, another for personal preferences, and a third for shared preferences with a partner.

This approach enables the system to adapt to varying situational needs, ensuring a more tailored and personalized user experience. By maintaining distinct sessions within a unified user profile, the system can provide recommendations that align with the specific context of each interaction.

3.5 User Interface

The user interface is designed to be intuitive and user-friendly, enabling seamless interaction with the system and facilitating user feedback.

To achieve this, the project employs Gradio [1] as the framework for the user interface. Gradio is a Python library that simplifies the creation of interactive and accessible user interfaces for machine learning models. Its flexibility and ease of customization make it an ideal choice for this project. Gradio was chosen for its suitability in creating interactive interfaces, particularly chatbots, alongside its ability to integrate with various libraries and frameworks. In contrast with other frameworks like Flask or Streamlit, Gradio offers a more straightforward approach to building user interfaces, especially for prototyping.

The user interface will include components that allow users to engage in conversations with the agent, request recommendations, and view both the generated recommendations and the extracted preferences derived from the conversation.

For what the user interface will look like, please refer to the User Interface Wireframe (Section 3.5.1) below.

3.5.1 User Interface Wireframe

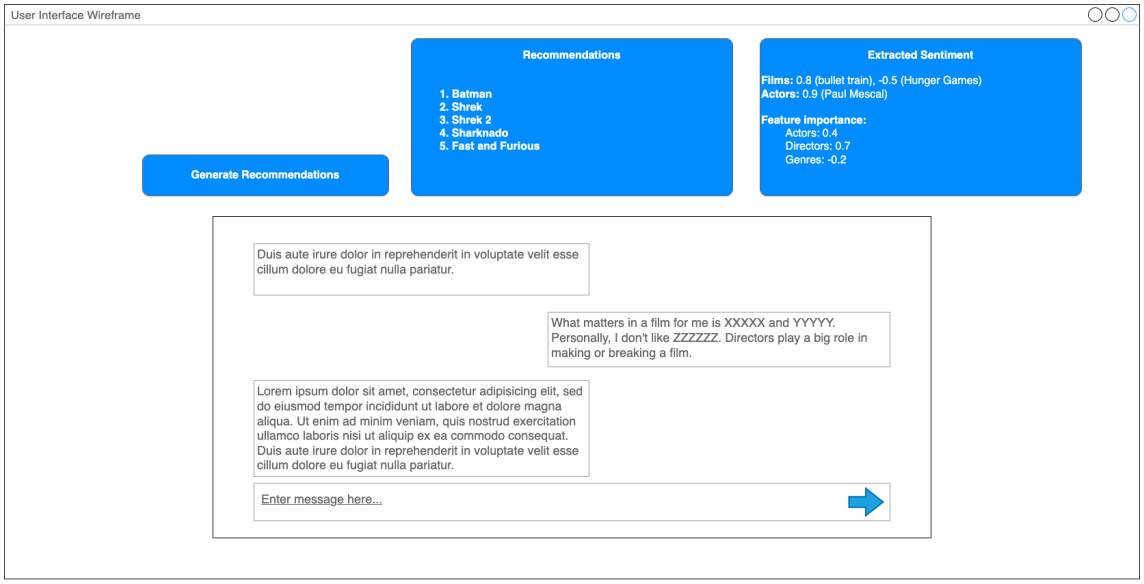

Figure 3.2: Intended User Interface Wireframe

The main components of the user interface are:

- **Chat Interface** - The chat interface allows the user to interact with the agent, providing a conversational experience. The user can ask questions and provide feedback on the recommendations.
- **Recommendations** - The recommendations are displayed in a list format, allowing the user to view the recommended films.
- **Extracted Preferences** - The extracted preferences are displayed in a list format, allowing the user to view the preferences that were extracted from their message.
- **Generate Recommendations Button** - The generate recommendations button allows the user to request recommendations based on their preferences.
- **Session and User IDs** - The session and user IDs are displayed at the top of the interface, allowing the user to view and alter which session they are in and which user profile they are using.

Chapter 4

Implementation

The system architecture design is visualised in Figure 3.1, which provides an overview of the system's components and their interactions. **To see a run through of the implemented system, see Section 6.1.**

4.0.1 Technologies and Languages

The system was implemented using Python 3.10. Python was chosen as the primary language for the project due to its extensive libraries and frameworks for data manipulation, machine learning, and natural language processing. Python is also well suited for rapid prototyping and development, which was a key requirement for this project.
Additionally, I wanted to learn more about Python, particularly in the areas of data science. This provided a great opportunity to do so.

The libraries and frameworks used in the system are listed below:

Library/Framework	**Purpose**
`pandas`	For data manipulation and analysis
`gradio`	For the user interface
`numpy`	For numerical computations
`LensKit`	For collaborative filtering
`FuzzyWuzzy`	For string matching
`sklearn`	For multi-label binarizer when processing the film data
`Ollama`	For running the LLM locally
`LangChain`	For LLM system integration

Table I: Libraries and Frameworks Used in the System

4.1 Recommendation System

As covered in the recommendation system Section 3.2, the recommendation system is designed to provide recommendations based on user preferences and feedback. It uses a combination of content-based filtering and collaborative filtering to provide recommendations.

4.1.1 Content-based Filtering

As there's no frameworks or libraries for content-based filtering with the qualities needed for recommendation and feature adjustment, a custom implementation was needed.

Importantly, this implementation is intended to be a **proof of concept**, and is not intended to be the best implementation. With this in mind, as mentioned in Section 3.2.1, the content-based filtering system uses the following features:

- Genres
- Directors
- Cast

More features could be added, such as year, but the listed features are sufficient for a substantial proof of concept. Adding anything more would be a stretch goal, and would not be necessary for the proof of concept. With this in mind, there is not much reason to evaluate the content-based filtering system in detail.

4.1.2 Collaborative Filtering

The collaborative filtering system is implemented using the `LensKit` library[**lenskit**], which provides a framework for building and evaluating recommender systems. It is a user-user collaborative filtering system, which uses the user-item matrix to find similar users and recommend items based on their preferences.

Again, as the recommendation system is only there to provide a working implementation of a recommendation system with key features, the collaborative filtering system is not the main focus of the project. If given more time, it would be great to implement a custom collaborative filtering system, with visibility into the neighbours profiles and the ability to adjust the weights of the neighbours based on their attributes.

For the collaborative filtering feature, the project uses the `UserUser` algorithm from the `user_knn` module that the `LensKit` library provides. The default `UserUser` parameters of `user_knn` with the number of nearest neighbours `nnbrs=15` and minimum neighbours `min_nbrs=3` parameters were used, as they provided a good balance between performance and accuracy.

4.2 Data Processing

4.2.1 Datasets

The project utilized two primary datasets: the TMDB dataset[10], sourced from Kaggle[3], and the MovieLens dataset[15]. Additionally, an ensemble dataset[4] combining TMDB and MovieLens data was used to enrich the film information with cast and director details.

The TMDB dataset provided detailed film information, while the MovieLens dataset was used for user ratings, which were essential for implementing the user-user collaborative filtering component of the recommendation system. The ensemble dataset was used to extract cast and director information to be used in the content-based filtering system.

4.2.2 Data Processing

The ensemble dataset was leveraged to create a CSV file containing the top three cast members and directors for each film.

To prepare the data for the recommendation system, the films were stripped of irrelevant metadata, retaining only the essential attributes. The cast and director information from the ensemble dataset was appended to the film data to enhance its utility for content-based filtering. Then, the genres were one hot encoded to create a binary representation of the genres, allowing for efficient filtering and matching.
This streamlined dataset was then used to power the recommendation engine, ensuring efficient and accurate recommendations.

4.3 User Interaction

The user interaction process was designed to be as simple and intuitive as possible, while still allowing for a comprehensive exploration of user preferences.

In the chat interface, the user can talk freely to the system, and the system will address the users requests with the appropriate tooling and features.

4.3.1 User Profile

The user profile is a key component of the system, as it stores the user's preferences and feedback. It holds the weighting scheme for the hybrid recommendation system alongside the content-based filtering system. It also holds the users feature item scores, which are used to provide recommendations based on the user's preferences.

It is represented as a dictionary, with the following structure:

```
{
    "user_id": {
        "userId": user_id,
        "weightings": {
            'cast_ft_weight': 0.3,
            'genre_ft_weight': 0.4,
            ...
        },
        "feature_profile": {
            'Quentin Tarantino': 0.5,
            'Action': 0.8,
            4995: 0.7, // film id
            390012: 0.6, // actor id
            ...
        }
    }
}
```

The user profile is designed to be flexible and extensible, allowing for the addition of new features and attributes as needed. If given more time, the user profile would have it's various features in the `feature_profile` dictionary separated into their own dictionaries, such as `cast_profile`, `genre_profile`, etc.

4.3.2 User Profile Creation

The user profile is created when the user first interacts with the system, and is populated with a set of default weightings. If there are film ratings provided, the system will use these ratings to populate the user profile with the relevant feature scores.

The system does this by going through the film ratings and extracting relevant features from the films dataset alongside the ensemble dataset. The system will then add the relevant features to the user profile, with a score of the rating given by the user multiplied by the weighting of the feature.

4.3.3 User Profile Score Adjustment

To adjust the user profile scores based on feedback, I used the hyperbolic tangent (`tanh`) function. This approach was chosen because `tanh` provides a smooth, non-linear mapping that naturally tapers values between 0 and 5, ensuring that extreme adjustments are avoided while still allowing meaningful changes.

The floor of 0 was implemented to ensure that films with a high relevance score in a category the user doesn't like, such as 3.4, do not get punished more than films that are less relevant, such as 0. The formula used for updating the scores, with a floor of 0, is as follows:

$$S' = \max\left(0, 5 \times \tanh\left(\frac{S + \alpha \cdot \Delta S}{5}\right)\right)$$

where:

- S is the current score.
- ΔS is the sentiment adjustment derived from user feedback.
- α is a scaling factor that controls the sensitivity of the adjustment.
- S' is the updated score.

The use of `tanh` ensures that:

- Scores remain bounded within a reasonable range, preventing runaway values.
- Adjustments have diminishing returns as scores approach their limits, reflecting a natural plateau effect.

For example, if a user expresses strong positive feedback about a film, the corresponding score in their profile is increased, but the adjustment becomes less pronounced as the score nears the upper limit of 5. Similarly, negative feedback reduces the score, but it is bounded by 0, ensuring that a film cannot be penalized below this threshold, for having relevance in a category the user does not like.
This design choice was made to ensure that the system remains stable and does not produce erratic behaviour in response to user feedback.

The scaling factor α was chosen based on the desired sensitivity of the feedback. The value of α can be adjusted to control how strongly feedback affects the profile. In the actual implementation, a value of $\alpha = 0.3$ was used, but this can be fine-tuned based on user testing and feedback if the system were to be changed in the future.
This approach provided a robust and intuitive way to manage user profile updates, **ensuring stability while still reflecting user preferences effectively.**

Matching Items in the User Profile

To ensure robust and accurate matching of items in the user profile, the system leverages the `FuzzyWuzzy` library[12], which employs the Levenshtein distance algorithm to calculate the similarity between strings. This approach allows the system to handle variations in input, such as typos, syntax errors, or slight differences in naming conventions, ensuring a seamless user experience.

The matching process is designed to prioritize efficiency and accuracy. Initially, the system performs a direct match search on the dataset to locate the item. If the item is not found, the system then performs a fuzzy search using `FuzzyWuzzy` to identify potential matches based on string similarity. This two-step approach minimizes computational overhead while maintaining high accuracy in matching.

This functionality is particularly critical in scenarios where user input may deviate from the exact titles or names stored in the dataset. For example, if a user mentions `"The Godfather"` but the dataset stores it as `"Godfather, The,"` the fuzzy matching algorithm ensures that the correct item is identified and processed.

By incorporating fuzzy matching, the system enhances its ability to adapt to user input dynamically, improving the overall reliability and robustness of the recommendation process. This design choice reflects the project's emphasis on creating a user-friendly and resilient recommendation system capable of handling real-world variability in user interactions.

4.4 Large Language Model Integration

Originally, the plan was to use a multi-model approach, where each model would be responsible for a specific task. For example, the conversational agent would be a conversational model, the routing would be done by an instruct model, as would the feedback extraction model.

However, after consulting with a former colleague from Genesys, the decision was made to pivot to a single large language model (LLM) approach. This decision was made to reduce the complexity of the system, and to allow for a more lightweight implementation.

The LLM used in this project is the `llama3.2:3b-instruct-q4_K_M` model[13], which is a quantized instruct based model. As it is a quantized model, it dramatically reduces the memory needed for the system, allowing it to run on local machines with limited resources. While this reduces the accuracy of the model, it is still able to perform well on the tasks needed for the project as high accuracy was not a key requirement.

This was a key factor in the decision to use this model, as it allows for the system to feasibly be deployed on a local machine.

Ollama was used to run the model, as it is a lightweight and easy to use framework for running LLMs. The LLM was used to handle various tasks, such as exploring user preferences, explaining recommendations, and extracting feedback.

The LLM was integrated into the system directly, with the system routing user queries to the appropriate prompt and passing the response back to function calls.

4.4.1 Prompts

The `llama3.2:3b` model card provided a good starting point for the prompts, but they were further refined to suit the specific needs of the project. The prompts used for the Large Language Model (LLM) were designed to handle various tasks, such as exploring user preferences, explaining recommendations, and extracting feedback. A detailed table of the prompts can be found in appendix A.2.

Below is a summary of the key prompts:

- **Film Chat Explore Prompt**: Designed to engage users in a conversation to understand their film preferences.
- **Explain Recommendation Prompt**: Provides an explanation for a recommendation based on user profile attributes.
- **Glean Feedback Prompt**: Extracts user preferences and sentiment from feedback messages.

The prompts were structured to ensure clarity and consistency in the responses generated by the LLM. The design of these prompts was important for the overall performance of the system.

The prompts directly influence the quality of the interactions and recommendations provided to users, where they were carefully implemented to ensure that the LLM could effectively understand and respond to user queries.

4.4.2 Prompt Engineering

The prompt engineering process proved to be challenging, as it required careful consideration of the language and structure used in the prompts. The model is quite liable to deviate from the given instructions.

Secondary Information Extraction

The model was also used to extract secondary information from the user input, such as the user's supposed preferences and sentiment. For example, if a user expresses a preference for the film *Fantastic Mr. Fox*, the system can infer a potential interest in other works by the same director, Wes Anderson, or films with a similar style or genre. This inference is achieved by having the model leverage its knowledge of features associated with the film, such as its director, cast, and genre, and adjusting the user's profile accordingly.

This **significantly expands the systems knowledge of the user**, and allows the system to provide more personalized and relevant recommendations, even when explicit preferences are not directly stated by the user. This is particularly useful in cases where the user may not be aware of their own preferences or may not have explicitly stated them. See Figure 4.1 for an example of this, where the user only mentions *The Big Short*, but the model infers that they probably like *Adam McKay* the director of the film, alongside *Christian Bale*, *Steve Carell*, and *Ryan Gosling* who are actors in the film.

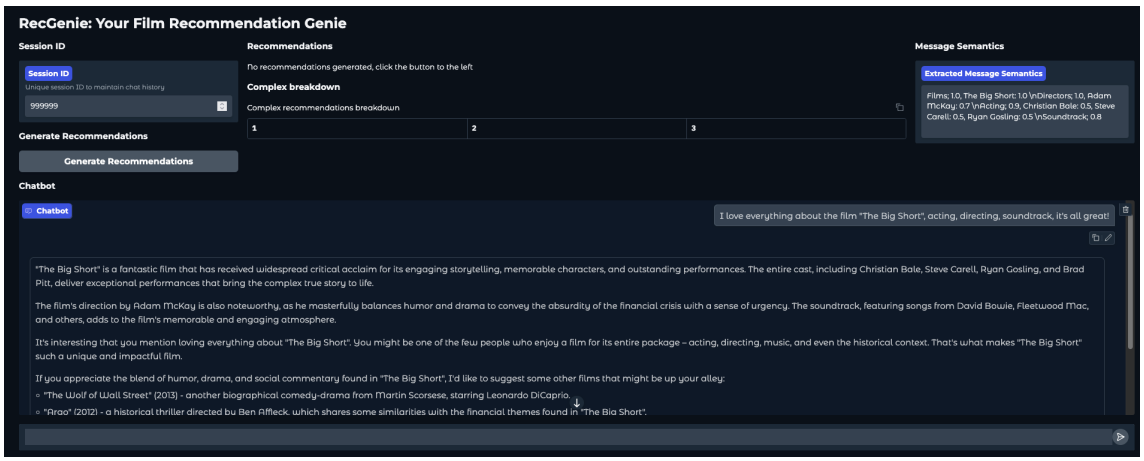

Figure 4.1: Example of secondary information extraction

Prompt Structure

Another challenge was ensuring that the LLM understood its instructions, but would not let the example responses influence the model's output. The "Glean Feedback Prompt" was particularly challenging, as it required the model to extract user preferences and sentiment from feedback messages, while also extracting related films and actors.

At times, if the user input was short, and did not contain much information, the model would sometimes included the example responses in the output, or hallucinate information that was not present in the user input. For example, it tended to include *Shawshank Redemption*, and the films director, *Frank Darabont* in the output, even if it was not mentioned in the user input, as it is one of the most well known films worldwide. See Figure 4.2 for an example of this.

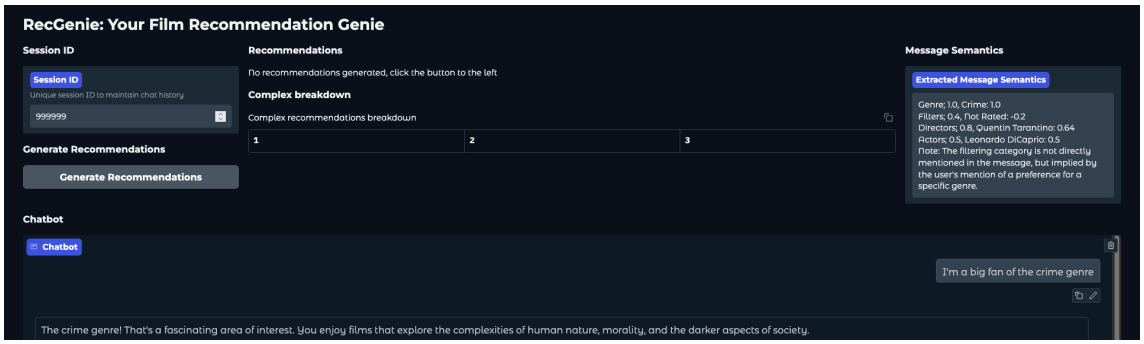

Figure 4.2: Example of LLM hallucination

Inference Latency

The inference latency of the LLM was a significant consideration, as it directly impacts the user experience. The initial response time for the LLM when generating sentiment analysis from a user message was around 35 seconds, which was too slow for the conversational system, even with the user taking time to read the agent response and to respond. A lot of this was due to the prompt structure being large and bulky, which caused the model to take longer to process the input and generate a response.

Solution Through iterative prompt engineering, the response time was significantly reduced to approximately 15 seconds, which is a substantial improvement with a **reduction of 57%** in response time. This improvement was achieved by optimizing the prompt structure, specifically by minimizing the number of tokens in the generated response.

There is more work to be done in this area, as the model is still quite slow at times, and the response time can vary significantly depending on the prompt. One idea would be to implement a mapping or flag system where specific prefixes are used to denote categories. For example; `F;` for films, `A;` for actors, `D;` for directors, and `G;` for genres. This would allow the system to quickly identify the type of item being referenced and parse it accordingly, while importantly reducing the number of tokens in the response.

4.5 Challenges Faced

The implementation of the system faced several challenges, particularly in the areas of prompt engineering, user interaction and data management. These challenges were addressed through a combination of iterative design, testing, and optimization, leading to a more robust and efficient system.

4.5.1 User Interaction

Maintaining a natural and engaging flow for the user in both conversation and generating recommendations was a significant challenge. For this, the system was designed to route the users queries to the appropriate prompts, ensuring there was a clear and consistent flow of information.

Initially, the routing was done using the LLM, where the model would determine which prompt

to use based on the user's input. However, this approach was slow and inefficient, as it required the model to process each query and determine the appropriate prompt. Additionally, the model would sometimes misinterpret the user's intent, leading to incorrect prompt selection.

This approach was replaced with a keyword-based routing system, where the system searches for specific keywords in the user's input and maps the query to the appropriate prompt. By implementing a dictionary-based mapping of keywords to prompts, the routing process became significantly faster and more efficient, reducing the reliance on the LLM for routing decisions and improving the overall accuracy of the system.

4.5.2 LLM Performance

The performance of the LLM presented a notable challenge, requiring careful consideration of the model's capabilities and constraints.

Although the selected model was lightweight and efficient, it initially required up to 35 seconds to generate sentiment analysis from a user message.

Through iterative prompt engineering, the response time was significantly reduced, with the average time decreasing to approximately 15 seconds. This was a substantial improvement with a **reduction of 57%** in response time.

A large part of this improvement was achieved by optimizing the prompt structure, specifically by minimizing the number of tokens in the generated response. To read more about the prompt engineering process, see Section 4.4.2 and Section 4.5.3. Additionally, the switch from the standard version of llama3.2 to the quantized version of llama3.2:3b-instruct-q4_K_M also contributed to the improved performance, as it reduced the memory needed, allowing it run faster with less memory pressure.

4.5.3 Prompt Engineering

The prompt engineering process was a significant challenge, as it required careful consideration of the language and structure used in the prompts. The model was sensitive to the smallest changes in the prompts, which could lead to drastically different responses.

One key challenge was ensuring that the LLM understood it's instructions, but would not let the example responses influence the model's output. The "Glean Feedback Prompt" in Section 4.4.1 was particularly challenging, as it required the model to extract user preferences and sentiment from feedback messages, additionally extracting related films and actors from the user input. At times, if the user input was short, and did not contain much information, the model would sometimes include the example responses in the output, or hallucinate information that was not present in the user input. See Section 4.4.2 for more details on this.

There is still a lot of work to be done in this area. There is still a lot of noise at times in the LLM outputs, and the model would sometimes forget or ignore the formatting instructions, which then impacted the performance of the system as it was not able to parse the output correctly.

Solution To mitigate this, fine-tuning the model on domain-specific data could improve the quality of interactions and reduce issues such as hallucinations or non-conforming outputs. This would require training the model on a custom dataset of preference extraction and sentiment analysis outputs, which would help the model learn the desired output format and improve its performance on these tasks. This would be costly and time-consuming, but the use of a smaller model to generate synthetic data could help reduce the cost and time needed to train the model.

4.5.4 Limited Data

The system was limited by the data available in the datasets used. The TMDB dataset provided a wealth of information, which was up to date, but the user ratings from the MovieLens dataset were limited up to October 2023. Even at that, a large proportion of the ratings were for films over 20 years old, which limited the relevance of the recommendations for users looking for more recent films. As such, when the user user collaborative filtering was used, the applicable films decreased from over 100,000 down to just over 28,000 films.

Solution To mitigate this, whenever the collaborative filtering failed to provide a score for a film, the system would take the average IMBD rating for the film, and multiply it by 0.4, as not to over penalise the film for not having other ratings. This was a simple solution, but allowing films that were not in the user ratings dataset to still be recommended to the user. However, this is certainly not a great solution.

The same issue was present with the credits dataset, where the latest film in the dataset was from 2018, over 7 years ago. This limited the ability of the system to provide collaborative filtering recommendations for films. Unfortunately, due to time constraints, there was no time to implement a solution for this.

4.6 Current Limitations

The current implementation of the system has several limitations that need to be addressed in future work and research. These limitations include:

- **Lack of Data:** The system is limited by the data available in the datasets used. This caused the recommendation system to be less effective. See Section 4.5.4 for more details.
- **Noise in Recommendations:** The LLM extracts data that was not mentioned by the user. This can lead to noise in the recommendations, as the system may recommend films that are not relevant to the user.
- **No Speech-to-Text or Text-to-Speech:** The system does not currently support voice-based interaction, which could enhance user experience, particularly for use cases like television-based systems.
- **No Fine-Tuning of LLMs:** The current implementation does not include fine-tuning of the LLM on domain-specific data, which could improve the quality of interactions and reduce issues such as hallucinations or irrelevant outputs.
- **Inference Latency:** The LLM's inference latency remains a challenge, particularly for real-time applications. While prompt engineering has improved response times, further optimization is needed to enhance user experience.
- **Cold Start Problem:** Although the system deals with the cold start problem much better than traditional recommendation systems, especially as the LLM can explore the users tastes quickly and efficiently, it still somewhat suffers from the cold start problem.
- **Limited Feature Set:** The current implementation relies on a limited set of features for content-based filtering. While this is sufficient for the proof of concept, expanding the feature set could improve the quality of recommendations.
- **User Profile Management:** The user profile management system is currently basic and could benefit from more advanced techniques for managing user preferences and feedback.

Chapter 5

Evaluation Methodology

This project took from the thinkings of both [17] and [18] to evaluate the system as a whole, and not just the individual components, where evaluation of the project will be done in three parts:

1. Evaluation of the Recommendation System, including Feedback and Adjustment of Recommendations
2. Assessment of the Large Language Model Integration
3. Evaluation of the System through User Feedback

5.1 Recommendation System Evaluation

As mentioned in Section 3.2, the recommendation system is designed to be a proof of concept, and is not intended to be the best implementation.

What the system is being evaluated on is:

- The system works reliably.
- That the system is able to provide recommendations that make sense.
- The recommendation breakdown is generated and is understandable.
- The system is able to provide recommendations based on the features.
- The system provides recommendations unique to user preferences.

5.2 Large Language Model Evaluation

The large language model implementation is being evaluated on the following:

- The agent can work with the recommendation system.
- The agent works with the user to explore their preferences.
- The agent can provide a breakdown of the recommendations with an explanation that is understandable.
- The agent can handle out of scope conversations and can redirect the conversation back to the task at hand.
- The system is timely and responsive.

5.3 User Evaluation

For the user evaluation, users will undertake tasks and provide feedback. This will provide an objective measure of the recommendations and subjective feedback.

The user questions will be designed to evaluate the system as a whole, alongside the recommendation system and the large language model. The user will be asked to undertake one of two scenarios, depending on their preferences.

5.3.1 User Scenarios

The user will be provided with one of three sample profiles, and will be asked to follow the scenario steps below.

1. The user will be asked to start with a template sample profile.
2. The system will provide a set of recommendations based on the user preferences.
3. The user will ask the system to provide a breakdown and explanation of the recommendations.
4. The user will provide feedback on the recommendations and the explanation.
5. The user will discuss with the agent what feature(s) are important to them in recommendations and generate a new set of recommendations based on those alterations.
6. The user will evaluate the given recommendations and provide feedback on the recommendations.
7. Finally, the user will provide feedback on the system, the recommendations, and the system as a whole.

5.3.2 User Survey

Using the following questions, users will be asked to provide feedback on the system, the recommendations, and the system as a whole

User Survey Questions
Does the system feel useful?
How cumbersome is it to use the system?
How well does the system understand your preferences?
How useful is the recommendations explanation?
Did the system reflect your change in preferences in the recommendations?
Would you use the system again?
What do you like about the system?
What do you dislike about the system?
What would you like to see added to the system?
What do you think of the system as a whole?
How would you rate the system's objective overall?

Table I: User Survey Questions

Chapter 6

Evaluation and Testing Results

6.1 System Run Through

This section provides a brief run through of the system, demonstrating how the system works and how it can be used to provide recommendations based on user preferences, alongside it's feedback and adjustment system.

6.1.1 User Profile Starting Template

The system allows users to start with a template profile - e.g. action and crime films, romcoms, etc. This is done by providing a set of film ids and their associated ratings.

The system will then use this template to generate a user profile, which can be adjusted based on user feedback and preferences. For this example, the user profile is set to a template profile of action and crime films, with a few films added to the profile. See Table I for the ratings used for this example.

Film Id	**Film Title**	**Rating Score**
710	Golden eye	4
1770	Michael Collins	5
374720	Dunkirk	4.7
339403	Baby Driver	4.7
318846	The Big Short	4.4
627	Trainspotting	4.9
27205	Inception	3.6
106646	The Wolf of Wall Street	4.5
1893	Star Wars: Episode I - The Phantom Menace	3.8
550	Fight Club	4.2
98	Gladiator	4

Table I: Sample User Ratings

6.1.2 Generated Recommendations

As soon as the system is started, the user is presented with a set of recommendations based on their profile. You can see the recommendations in the image below Figure 6.1.

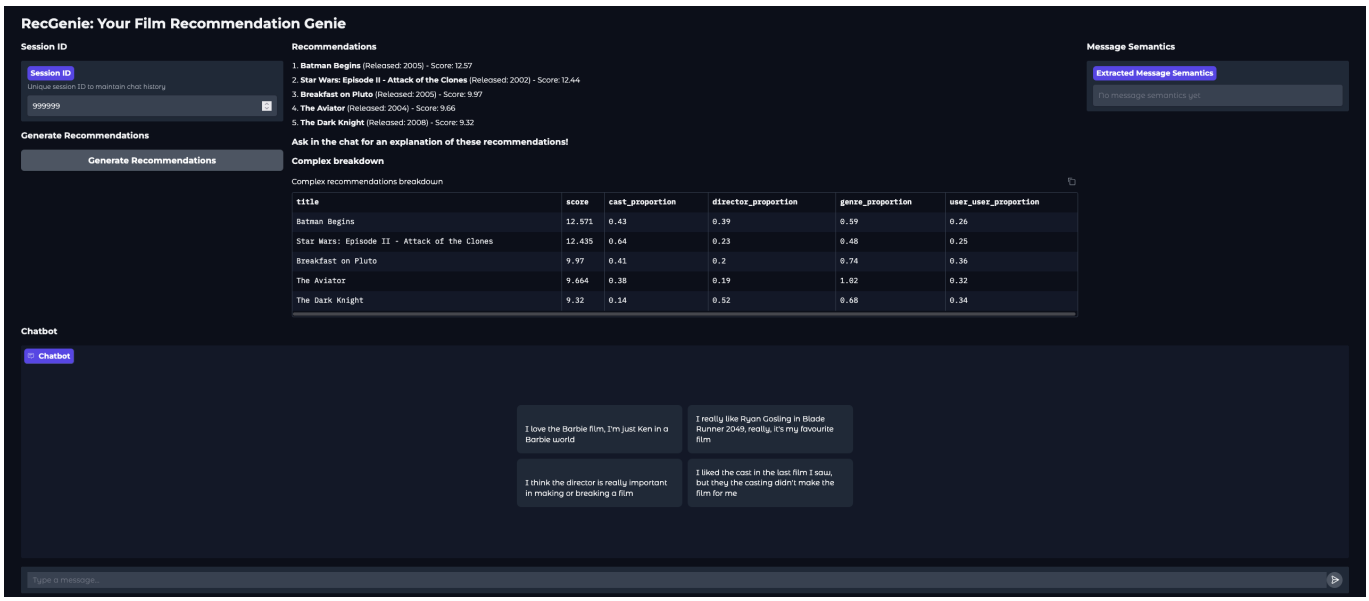

Figure 6.1: Sample Recommendations

6.1.3 User Interaction

The user can then interact with the system, providing feedback on their preferences and the recommendations.

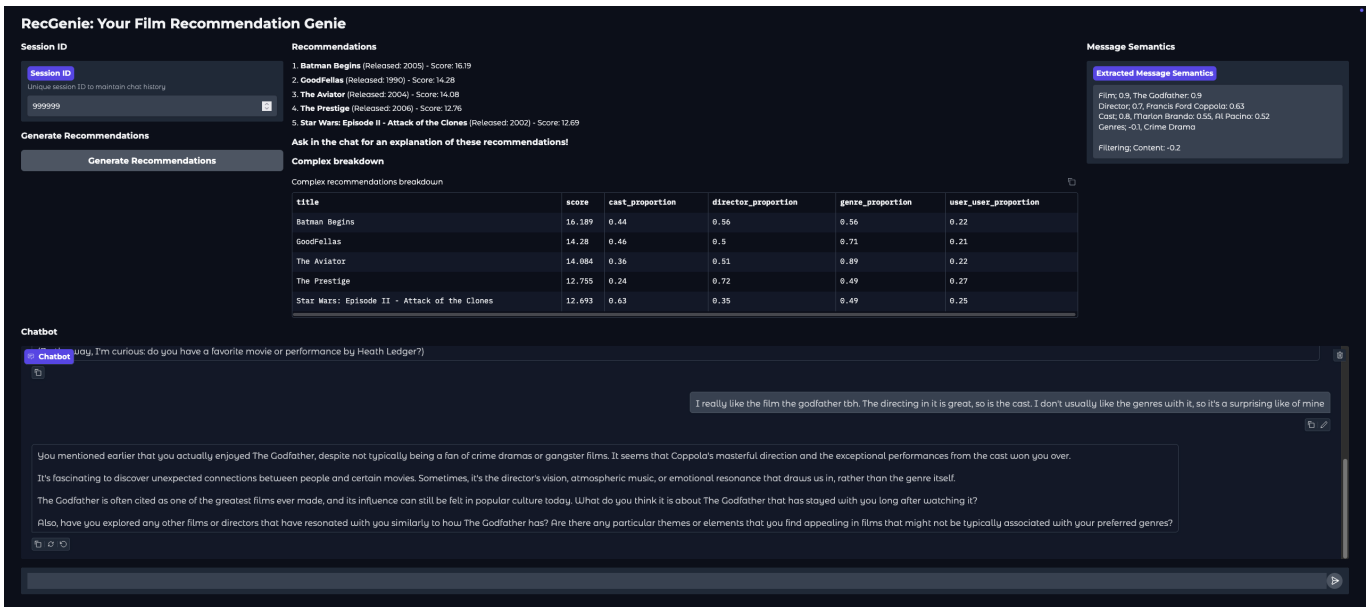

Figure 6.2: Sample User Interaction

6.1.4 New Recommendations Based on User Feedback

The user can also then provide feedback on the recommendations, and have the system adjust the recommendations based on their feedback, which you can see the extracted user preferences in the top right of the image.

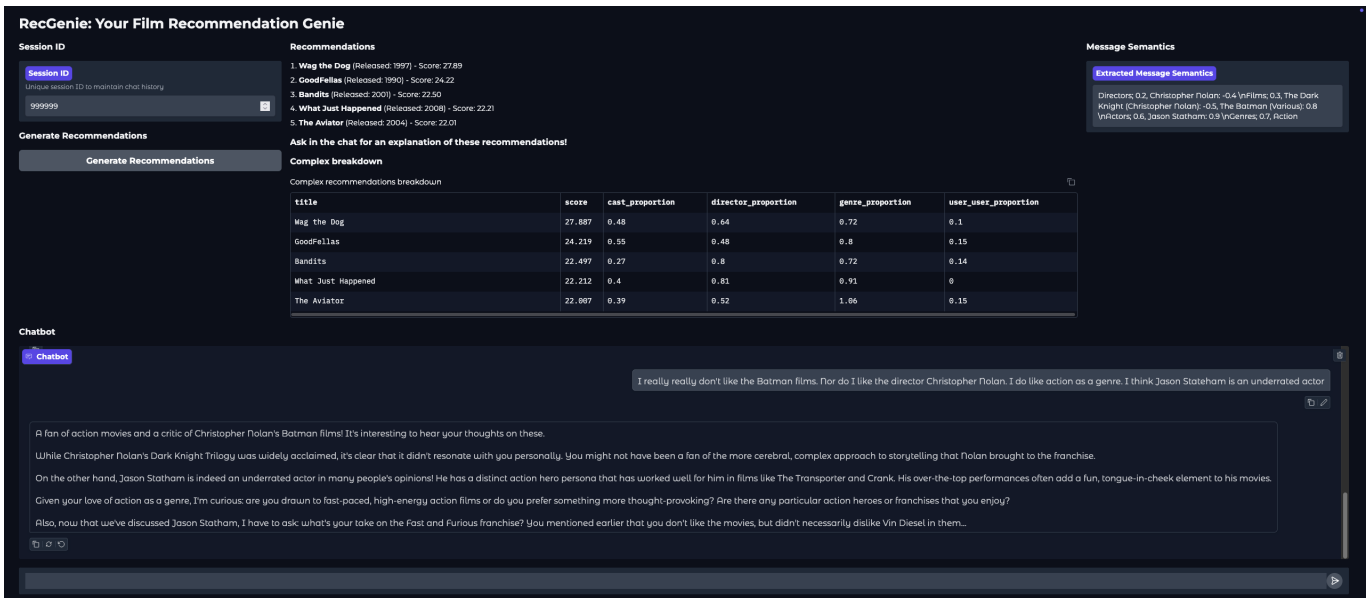

Figure 6.3: Sample User Feedback

After the user provides their feedback detailing their dislike for the film "Batman Begins", the system adjusts the user profile and generates new recommendations based on the updated profile. You can then see that the system has adjusted the recommendations based on the user feedback, and Batman is no longer in the top five recommendations.

6.1.5 Recommendation Breakdown

The user can also ask the system to provide a breakdown of a recommendation, which will show the user how the recommendations were generated. Here, the user asks the system to explain why it recommended "Batman Begins" to them, and the system is able to provide a detailed explanation of the recommendation.

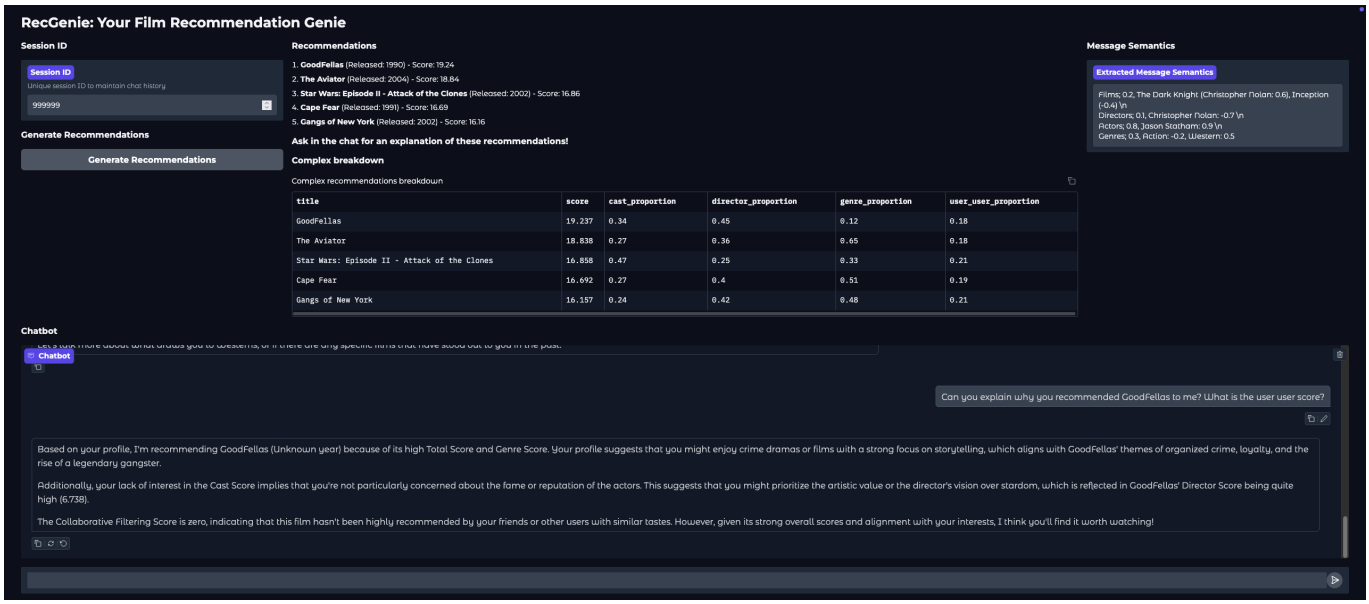

Figure 6.4: Sample Recommendation Breakdown

Summary The system demonstrates great performance in the subsections above, effectively **meeting all its goals as outlined in chapter 5**. It provides accurate, relevant, and explainable recommendations, adapts seamlessly to user preferences, and offers an intuitive and responsive conversational interface.

The system also demonstrates the ability to **provide relevant recommendations based on user preferences**, and is able to adjust the recommendations based on user feedback. Additionally, the integration of the large language model allows for it to employ immediate suggestions and thought provoking questions to the user, helping them explore their preferences alongside **greatly reducing the cold start problem.**

6.2 Large Language Model Evaluation

The large language model is able to provide a conversational interface that allows the user to explore their preferences and provide feedback. It is able to provide detailed responses to user queries, explore user preferences and keep the user engaged in the conversation. The system is able to provide a breakdown of the recommendations and explain why it recommended a certain film to the user.

6.2.1 Conversational Interface

The system is able to provide a conversational interface that allows the user to explore their preferences and provide feedback. It is easily able to provide detailed responses to user queries, explore user preferences and keep the user engaged in the conversation. Additionally, as mentioned in Section 6.1, the knowledge of the large language model allows it to employ immediate suggestions and thought provoking questions to the user, helping them explore their preferences alongside alleviating the cold start problem.

For example, in the following conversation, the system engages with the user and is able to talk about the directors style and asks the user about their opinions on other directors and genres.

```
User: While sometimes I do like a film I can switch my brain off
for, I do enjoy ones with excellent plot development.
For example, I think Andrei Tarkovsky is one of the greatest
directors of all time.
```

```
Assistant: You're looking for films that challenge you
intellectually and engage your critical thinking skills. You
appreciate complex storytelling, layered themes, and character
development.

Andrei Tarkovsky's films are indeed a great example of this type of
cinema! His slow-burning, contemplative style often rewards close
attention and multiple viewings. Films like "Solaris",
"Andrei Rublev", and "Nostalghia" are masterclasses in atmospheric
tension, philosophical themes, and visual storytelling.

Tarkovsky's work is not for everyone, I'm sure! His films often
require a willingness to slow down, observe, and reflect on the
emotions and ideas presented. But when you're able to immerse
yourself in his world, it can be incredibly rewarding.

Given your appreciation for thought-provoking cinema, I'd like
to explore some other directors or films that might appeal to you.
Are there any other directors or genres you've enjoyed in the past?
For example:
- French New Wave (e.g., Truffaut, Godard)?
- Italian Neorealism (e.g., De Sica, Visconti)?
- Eastern European art-house cinema?
- Independent dramas or thrillers?

Or perhaps we can explore some specific films that might fit your
taste for complex storytelling and intellectual engagement?
```

6.2.2 User Preference Extraction

The system is able to extract user preferences from the conversation and use them to provide recommendations. For example, in the following conversation, the user is able to provide their preferences and the system is able to extract them and use them to provide recommendations:

Input: *I really like Denis Villeneuve films, they're excellent. If I had to pick an actor, I would say Leonardo DiCaprio is my favourite. I prefer my recommendaitons to be based off content filtering.*					
Category	**Category Score**	**Item 1**	**Score**	**Item 2**	**Score**
Films	0.9	Arrival	0.9	Blade Runner 2049	0.8
Directors	0.8	Denis Villeneuve	0.9		
Actors	0.4	Leonardo DiCaprio	0.5		
Filtering	N/A	Content Filtering	0.3		

Table II: Semantic Breakdown of Inputs

Input: *Also, the films snatch and lock stock and two smoking barrells really have a special place in my heart. They are such great films*					
Category	**Category Score**	**Item 1**	**Score**	**Item 2**	**Score**
Films	0.9	Snatch	0.7	Lock, Stock and Two Smoking Barrels	0.8
Directors	0.6	Guy Ritchie	0.55		
Actors	0.4	Vinnie Jones	0.6	Jason Flemyng	0.5
Genres	0.7	Crime Comedy	0.75		

Table III: Semantic Breakdown of Inputs

6.2.3 Recommendation Breakdown

As you can see in Section 6.1, the system is able to provide a breakdown of the recommendations and explain why it recommended a certain film to the user. For example, in the following conversation, the user asks the system to explain why it recommended "Batman Begins" to them, and the system is able to provide a detailed explanation of the recommendation:

```
User: Explain please, why you recommended Batman to me.
```

```
Assistant: Based on your profile, I recommend "Batman Begins"
because of its impressive overall score (12.571) and a
particularly high Director Score (4.88). This suggests that you
appreciate well-crafted storytelling and direction, which are
often hallmarks of Christopher Nolan's films. Additionally, the
high Genre Score (7.462) indicates that "Batman Begins" is part
of the action/adventure genre, which might be up your alley
given your profile.

While the Collaborative Filtering Score is
lower, suggesting that not many in your network have seen this
film, I'm making an educated guess that you might enjoy it based
on its high scores across other attributes. Overall, "Batman
Begins" seems like a well-rounded and engaging film that aligns
with many of your preferences, which is why I recommend giving
it a try!
```

A screenshot of the recommendation breakdown is shown in Figure A.1.

6.2.4 Responsiveness

The system was able to respond in a timely manner and was able to handle out of scope conversations and redirect the conversation back to the task at hand.

On average, the system was able to start responding to user prompts in under 7 to 10 seconds, even with long prompts full of user preferences and associated comments when the user was providing feedback.

In the case of recommendation explanations, the system was able to provide a detailed explanation of the recommendation in under 10 seconds.

6.3 User Evaluation Results

The user evaluation was conducted with 4 participants, who were asked to complete a series of tasks and provide feedback on the system. The evaluation aimed to assess the system's usefulness, ease of use, ability to understand user preferences, and overall potential. The process is detailed in Section 5.3, and the full survey results can be found in appendix A.4.

6.3.1 Survey Results

The survey results were overwhelmingly positive, with users highlighting the system's ability to provide relevant recommendations, adapt to changing preferences, and explain its reasoning. Below is a breakdown of the key findings:

System Usefulness: Users rated the system highly in terms of usefulness, with three users giving it a score of 5 and one user giving it a 4. Comments emphasized the system's intuitive nature and its ability to provide reasoned and relevant recommendations. For example, one user stated, **"The system is a lot more intuitive than Netflix is."** Another user noted the system's ability to remove a disliked film from recommendations after expressing their distaste for its director and genre.

Ease of Use: The system was rated as easy to use, with scores ranging from 1 (not at all cumbersome) to 5 (very easy to use). Users appreciated the conversational interface and the simplicity of the prompts. One user remarked, **"It's very easy to use because of the conversational interface with the system."** Another user highlighted the straightforward nature of the prompts, making the system accessible even for first-time users.

Understanding User Preferences: The system demonstrated a strong ability to understand and adapt to user preferences, with three users rating it a 5 and one user rating it a 4. Users praised the system's responsiveness and its ability to incorporate their preferences into recommendations. For instance, one user commented, **"It really understood my interests and took them into account while recommending."** Another user noted that while some noise was present in the recommendations, the system was highly responsive when relevant filters were applied.

Recommendation Explanations: The system's ability to explain its recommendations was well-received, with scores of 4 and 5 across the board. Users found the explanations clear and concise, though one user suggested adding more detail about user-user similarity, such as a summary of the profiles of similar users. As mentioned in Section 3.2.2, this feedback highlights an opportunity to enhance the transparency of the collaborative filtering process.

Adaptability to Changing Preferences: All users confirmed that the system successfully reflected their changing preferences in its recommendations. For example, one user noted, **"The system refreshed its recommendations after each preference,"** while another highlighted how the system removed a disliked film from the recommendations after feedback.

Overall Potential: Users rated the system's potential highly, with three users giving it a score of 5 and one user giving it a 4. They appreciated the system's concept, execution, and ability to provide a conversational, adjustable, and explainable recommendation experience. One user described the system as **"very promising,"** and another stated, **"I would definitely use this system if it was packaged into a more accessible platform (e.g., an app or website)."**

6.3.2 User Feedback

The feedback provided by users was instrumental in identifying the system's strengths and areas for improvement.

Positive Feedback: Users praised the system for its intuitive interface, ability to adapt to preferences, and clear explanations. They found the system engaging and appreciated its ability to provide high-quality recommendations. One user described the system as "**executed very skillfully,**" while another highlighted its **"vast knowledge and accurate recommendations"**.

Suggestions for Improvement: Users provided constructive feedback for enhancing the system:

- **Visual Enhancements:** Users suggested including movie posters alongside recommendations to make the interface more visually appealing.
- **User-User Similarity Details:** One user requested a summary or background information about similar users used in collaborative filtering to improve transparency.
- **Suggested Replies:** Another user recommended adding suggested replies under the chatbot to reduce typing effort.
- **Noise Reduction:** While the system was praised for its responsiveness, some users noted occasional noise in the recommendations and suggested refining the filtering process.

6.3.3 Conclusion

The user evaluation results demonstrate that the system is effective in providing conversational, adjustable, and explainable recommendations. Users found the system engaging, intuitive, and capable of adapting to their preferences.

Chapter 7

Conclusion

7.1 Conclusion

The project successfully demonstrated the potential of integrating large language models (LLMs) with recommendation systems to create a conversational, adjustable, and explainable recommendation system. By combining collaborative filtering, content-based filtering, and conversational AI, the system addressed key limitations of traditional recommendation systems, such as lack of personalization, transparency, and user control.

The user evaluation results highlighted the system's strengths, including its ability to provide relevant recommendations, adapt to user preferences, and explain its reasoning in a clear and concise manner. Users found the system intuitive, engaging, and effective in meeting their needs. The modular design of the system ensured flexibility, allowing for the addition of new features and the adjustment of existing ones.

Despite its success, the project also revealed areas for improvement, with further research and development needed. Challenges such as occasional noise in recommendations, limited user-user similarity explanations, and inference latency in the LLM were identified. These challenges provide valuable insights for future work and opportunities to further enhance the system.

Overall, the project demonstrated the **feasibility and effectiveness of combining LLMs with recommendation systems, paving the way for more user-centric and transparent recommendation technologies.**

7.2 Key Findings and Contributions

The project made several key contributions to the field of recommendation systems and conversational AI:

- **Integration of LLMs:** The project successfully integrated LLMs into a recommendation system, demonstrating their potential to enhance user interaction and provide personalized recommendations.
- **Modular Design:** The modular architecture allowed for flexibility and scalability, enabling the integration of various components and the addition of new features.
- **Explainability:** The system provided clear and concise explanations for its recommendations, addressing a common limitation in traditional recommendation systems.
- **User-Centric Approach:** The project emphasized user control and personalization, allowing users to adjust their preferences and receive tailored recommendations.
- **Real-World Application:** The project demonstrated the practical application of LLMs in a real-world context, showcasing their potential to enhance user experiences in various domains.
- **Challenges and Solutions:** The project identified key challenges in implementing LLMs in recommendation systems, such as inference latency and noise in recommendations, and proposed solutions to address these issues.

7.3 Reflections

The project was a rewarding experience, providing an opportunity to design and implement a system from the ground up. It allowed for the exploration of cutting-edge technologies, such as LLMs, and the integration of these technologies into a cohesive and functional system.

Additionally, the challenges faced during implementation, such as prompt engineering and data processing, provided valuable learning experiences and highlighted the complexities of working with LLMs.

The project also underscored the potential of modular system design, which enabled flexibility and scalability. This approach not only facilitates the integration of various components but also provides the foundations for future enhancements and extensions.

7.4 Future Work

While the project achieved its primary objectives, there are several areas for future exploration and improvement:

7.4.1 Enhancing Explainability

Users expressed interest in more detailed explanations of user-user similarity in collaborative filtering. Future work could focus on providing richer insights into the profiles of similar users, enhancing transparency and trust in the system.

7.4.2 Improving Noise Filtering

Some users noted occasional noise in the recommendations. Refining the filtering process and incorporating advanced techniques, such as sentiment analysis or contextual filtering, could improve the quality of recommendations.

7.4.3 Expanding Features

The system could be extended to include additional features, such as:

- **Visual Enhancements:** Adding movie posters or visual elements to make the interface more engaging.
- **Suggested Replies:** Providing suggested replies in the chatbot to reduce typing effort and improve usability.
- **Speech-to-Text and Text-to-Speech:** Enabling voice-based interaction for a more natural user experience, particularly for use cases like television-based systems.

7.4.4 Optimizing LLM Performance

The inference latency of the LLM remains a challenge. Future work could explore techniques such as prompt optimization, model fine-tuning, or the use of more efficient LLM architectures to reduce response times further.

7.4.5 Addressing the Cold Start Problem

The system could be enhanced to better address the cold start problem by leveraging synthetic data generation or pre-trained user profiles. This would improve the system's ability to provide recommendations for new users with limited interaction history.

7.4.6 Exploring New Domains

While the system focused on film recommendations, the modular design allows for its application to other domains, such as e-commerce, music, or books. Future work could explore the adaptability of the system to these areas.

7.4.7 Fine-Tuning LLMs

Given more time and resources, fine-tuning the LLM on domain-specific data could improve the quality of interactions and reduce issues such as hallucinations or irrelevant outputs.

7.4.8 Scalability and Deployment

The system could be deployed as a web application or mobile app, making it more accessible to a broader audience. This would involve addressing scalability challenges and optimizing the system for real-world usage.

By addressing these areas, the system could evolve into a more robust, versatile, and user-friendly recommendation platform, further advancing the state of conversational recommendation systems.

Appendix A

Appendix

A.1 User Scenarios Ratings

Film Id	Film Title	Rating Score
18785	The Hangover	3.4
67913	The Guard	3.7
40807	50/50	4
70160	The Hunger Games	2.4
77930	Magic Mike	4.5
72190	World War Z	3
187017	22 Jump Street	4
198663	The Maze Runner	4.1
207703	Kingsman: The Secret Service	3.5
99861	Avengers: Age of Ultron	2
167073	Brooklyn	4.2
254470	Pitch Perfect 2	4.3
271718	Trainwreck	4
314365	Spotlight	2.3
324786	Hacksaw Ridge	2
330459	Rogue One: A Star Wars Story	2
339846	Baywatch	3.9

Table I: Scenario 1: Sample User Profile

Film Id	Film Title	Rating Score
324849	The Lego Batman Movie	5
283995	Guardians of the Galaxy Vol. 2	5
324852	Despicable Me 3	5
99861	Avengers: Age of Ultron	4
330459	Rogue One: A Star Wars Story	2.94
10315	Fantastic Mr. Fox	5
141052	Justice League	2.5
284053	Thor: Ragnarok	4

Table II: Scenario 2: Children's Films User Profile

Film Id	Film Title	Rating Score
710	Golden eye	4
1770	Michael Collins	5
374720	Dunkirk	4.7
339403	Baby Driver	4.7
318846	The Big Short	4.4
627	Trainspotting	4.9
27205	Inception	3.6
106646	The Wolf of Wall Street	4.5
1893	Star Wars: Episode I - The Phantom Menace	3.8
550	Fight Club	4.2
98	Gladiator	4

Table III: Scenario 3: Crime, Comedy, Action User Profile

A.2 Prompts

Prompt Name	Prompt Description
Film Chat Explore Prompt	You're an assistant who's good at talking to users to find out about their film interests and what matters to them in a film.
Explain Recommendation Prompt	You are talking to the user, briefly explain the recommendation for the given film. These are all attributes based off the user's profile. Film Recommendation: Title: {title} ({release_date}) Total Score: {score} Cast Score: {cast_score} Director Score: {director_score} Genre Score: {genre_score} Collaborative Filtering Score: {user_user_score} Explain why this movie is recommended based on the given scores.

Table IV: Film Chat Explore and Explain Recommendation Prompt

Prompt Name	Prompt Description
Glean Feedback Prompt	Extract film preferences, filtering preferences, and sentiment scores from this user message. FORMAT THE OUTPUT WITH CATEGORIES AND SCORES LIKE THIS EXAMPLE, BUT ONLY USE ITEMS ACTUALLY MENTIONED OR IMPLIED IN THE USER MESSAGE BELOW: RULES: 1. Each category score shows how important this category is to the user (0.1-1.0). 2. Each item score shows how much the user likes that specific item (0.1-1.0). 3. IMPORTANT: IGNORE THE EXAMPLE ITEMS AND ONLY EXTRACT ITEMS FROM THE USER MESSAGE. 4. Also include implied items by name not directly mentioned but associated with mentioned items: a. For mentioned films, include their directors with a score of 60% of the film's score. b. For mentioned films, include their main actors with a score of 50% of the film's score. c. For mentioned films, include their genres with a score of 50% of the film's score. d. For mentioned directors, include their notable films with a score of 70% of the director's score. e. For mentioned actors, include their notable films with a score of 60% of the actor's score. 5. FORMAT MUST BE EXACTLY: 'Category; score, Item1: score, Item2: score \n'. 6. No explanations or additional text - only the structured data. 7. Use negative scores (-0.1 to -0.5) for things the user explicitly dislikes or has negative sentiment about. 8. IF AN ITEM IS MENTIONED BUT NO CLEAR SENTIMENT IS GIVEN, DEFAULT TO 0.3. 9. ENSURE EVERY ITEM HAS A NUMERIC SCORE. 10. For implied items, multiply the sentiment score of the source item by the relationship factor. 11. ENSURE ALL EXPLICITLY MENTIONED FILMS, DIRECTORS, ACTORS AND GENRES ARE INCLUDED, even if sentiment is negative.

Table V: Glean Feedback Prompt

A.3 Additional Figures

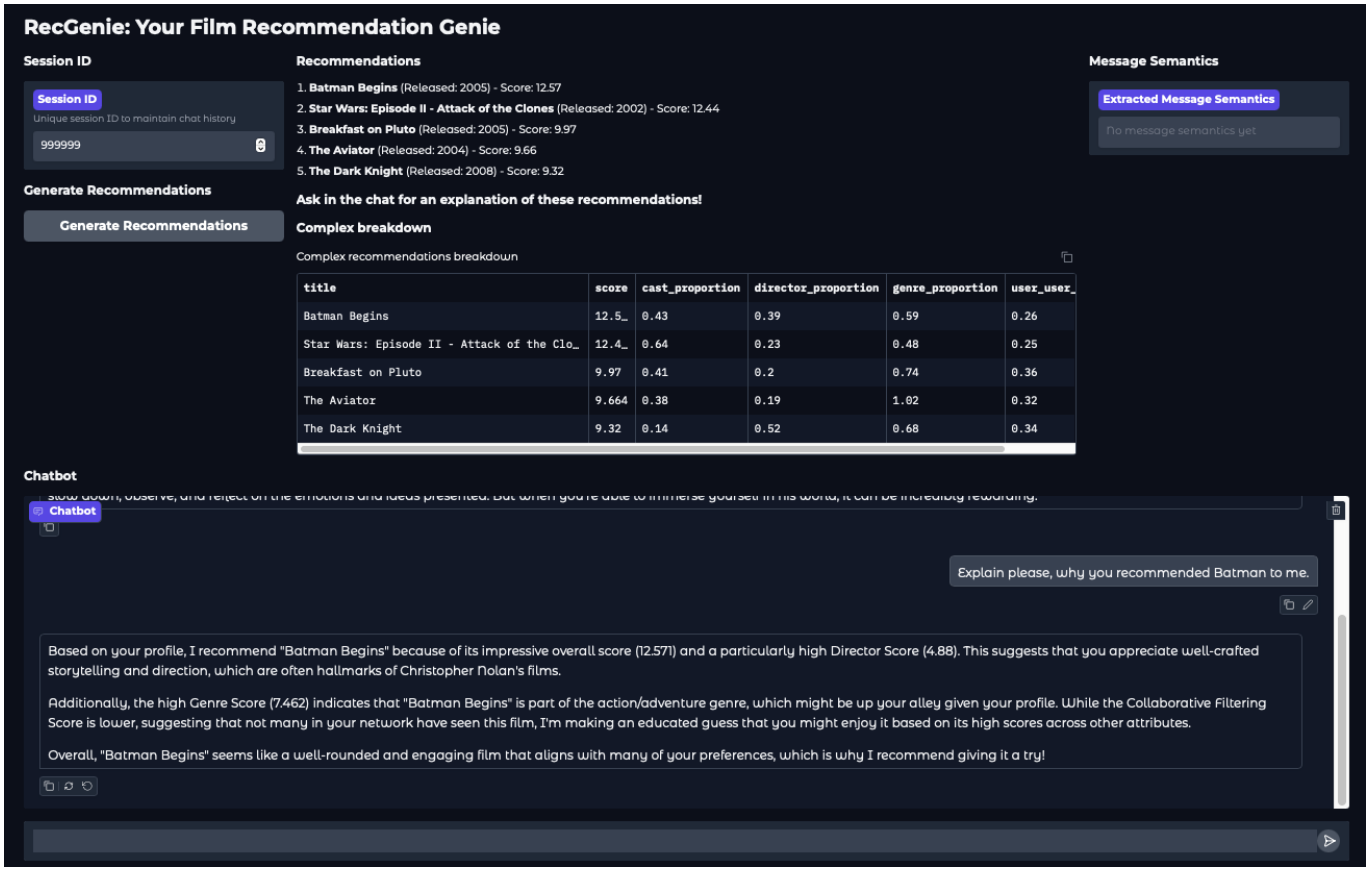

Figure A.1: Batman Score Explanation

A.4 Survey Results

Table VI: Survey Responses

Question	User1	User2	User3	User4
Which sample template	Option 2 - children's films	Option 3 - action and thriller	Option 3 - action and thriller	Option 1 - comedy and romcom
Does the system feel useful? (1 = not at all useful, 5 = quite useful)	5	5	4	4
Comments on the systems usefulness	I think the system is a lot more intuitive than other recommendation systems	N/A	answers took information from inputs and returned reasoned answers	The system shows some very promising movie recommendation. I was impressed by its knowledge of films and the speed of its response. It correctly removed a film from the recommendations after I expressed my distaste for its director and genre.
How cumbersome is it to use the system? (1 = not at all cumbersome, 5 = quite cumbersome)	1	2	1	2
Comments on how cumbersome the system is	It was very easy to use	N/A	prompts are easy to understand and complete	Very easy to use because of the conversational interface with the system.
How well does the system understand your preferences? (1 = very little, 5 = quite well)	5	5	4	5
Comments on the system understanding and learning	It really understood my interests and took them into account while recommending	N/A	words and meanings were interpreted and solid suggestions based on defined filters presented - some noise was present but if that was discarded the system was very responsive	It correctly recommended me some films based on my genre, director and actor preferences
How useful is the recommendations explanation? (1 = not at all useful, 5 = quite useful)	5	4	4	4
Comments on the usefulness	The system was clear and concise in explaining its choices	N/A	very solid recommendations	I would appreciate further clarity on the user-user similarity: something like a summary of the profile of the other users I am similar to?

Question	User1	User2	User3	User4
Did the system reflect your change in preferences in the recommendations?	Yes	Yes	Yes	Yes
Comments on the system reflecting your change in preferences	The system refreshed its recommendations after each preference	After some direction about preferences for older directors and films, goodfellas was suggested which was in line with what I thought was a good recommendation	yes as the system was progressing recommendations were updated to reflect my progress and chat history	It correctly removed a film from the recommendations after I expressed my distaste for its director and genre.
How would you rate the systems objective to provide a conversational, adjustable and explainable recommendation system? (1 = Very dissatisfied 5 = Very satisfied)	5	3	5	5
How highly would you rate the systems potential? (1 = no potential, 5 = a lot of potential)	5	4	5	5
Would you use the system again?	Yes I would	yes	yes - it has a very solid function and with some tweaking it has potential for film recommendations	Tá
What do you like about the system?	I think its concept is cool and its executed very skilfully	free form interface and quality responses and speed of processing	say to use and quick educated response	Ease of use, conversational and friendly interface, vast knowledge and accurate recommendations
What do you dislike about the system?	Theres some noise in the output especially if you don't go into detail on preferences	the user interface	some noise and unrelated data that needed to be ignored - can be easily filtered to focus on correct information	Lack of clarity on user-user similarities.
What would you like to see added to the system?	The bee movie, as it does not know the film	movie posters	N/A	Suggested replies under the chatbot to reduce the amount of typing the user has to do
What do you think of the system as a whole?	I think its concept is cool and its executed very skilfully	excellent feels like magic	I think it has valuable potential as a film recommender	The recommendation system is very promising. I would definitely use this system if it was packaged into a more accessible platform (e.g. an app or website)

Bibliography

[1] Abubakar Abid et al. "Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild". In: arXiv preprint arXiv:1906.02569 (2019).

[2] Tamim Mahmud Al-Hasan et al. "From Traditional Recommender Systems to GPT-Based Chatbots: A Survey of Recent Developments and Future Directions". In: Big Data and Cognitive Computing 8.4 (2024). ISSN: 2504-2289. DOI: 10.3390/bdcc8040036. **available at**: https://www.mdpi.com/2504-2289/8/4/36.

[3] Asaniczka and themoviedb.org. Full TMDB Movies Dataset 2024 (1M Movies). 2025. DOI: 10.34740/KAGGLE/DSV/11295902. **available at**: https://www.kaggle.com/dsv/11295902.

[4] Rounak Banik. The Movies Dataset. https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset. Specifically referencing the credits.csv file, which contains cast and crew information sourced from the TMDb API. 2017.

[5] Vincenza Carchiolo et al. "Searching for experts in a context-aware recommendation network". In: Computers in Human Behavior 51 (2015). Computing for Human Learning, Behaviour and Collaboration in the Social and Mobile Networks Era, pp. 1086–1091. ISSN: 0747-5632. DOI: https://doi.org/10.1016/j.chb.2015.03.028. **available at**: https://www.sciencedirect.com/science/article/pii/S0747563215002186.

[6] Yupeng Chang et al. "A Survey on Evaluation of Large Language Models". In: ACM Trans. Intell. Syst. Technol. 15.3 (Mar. 2024). ISSN: 2157-6904. DOI: 10.1145/3641289. **available at**: https://doi.org/10.1145/3641289.

[7] Harrison Chase. LangChain. 2022. **available at**: https://github.com/langchain-ai/langchain.

[8] Lei Chen and Meimei Xia. "A context-aware recommendation approach based on feature selection". In: Applied Intelligence 51 (2020), pp. 865 –875. **available at**: https://api.semanticscholar.org/CorpusID:225211792.

[9] Li Chen, Dongning Yan, and Feng Wang. "User perception of sentiment-integrated critiquing in recommender systems". In: International Journal of Human-Computer Studies 121 (2019). Advances in Computer-Human Interaction for Recommender Systems, pp. 4–20. ISSN: 1071-5819. DOI: https://doi.org/10.1016/j.ijhcs.2017.09.005. **available at**: https://www.sciencedirect.com/science/article/pii/S1071581917301313.

[10] The Movie Database. The Movie Database. 2024. **available at**: https://www.themoviedb.org/.

[11] Michael D. Ekstrand. "LensKit for Python: Next-Generation Software for Recommender Systems Experiments". In: Proceedings of the 29th ACM International Conference on Information and Knowledge 2020. DOI: 10.1145/3340531.3412778. arXiv: 1809.03125 [cs.IR].

[12] Fuzzy Wuzzy Search. **available at**: https://pypi.org/project/fuzzywuzzy/#description.

[13] Aaron Grattafiori et al. The Llama 3 Herd of Models. 2024. arXiv: 2407.21783 [cs.AI]. **available at**: https://arxiv.org/abs/2407.21783.

[14] Shuyu Guo et al. "Towards Explainable Conversational Recommender Systems". In: Proceedings of the 46th Inter SIGIR '23. ACM, July 2023, 2786–2795. DOI: 10.1145/3539618.3591884. **available at**: http://dx.doi.org/10.1145/3539618.3591884.

[15] F. Maxwell Harper and Joseph A. Konstan. "The MovieLens Datasets: History and Context". In: ACM Trans. Interact. Intell. Syst. 5.4 (Dec. 2015). ISSN: 2160-6455. DOI: 10.1145/2827872. **available at**: https://doi.org/10.1145/2827872.

[16] Cheng-Yu Hsieh et al. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data 2023. arXiv: 2305.02301 [cs.CL]. **available at**: https://arxiv.org/abs/2305.02301.

[17] Chen Huang et al. Concept – An Evaluation Protocol on Conversational Recommender Systems with System-ce 2024. arXiv: 2404.03304 [cs.CL]. **available at**: https://arxiv.org/abs/2404.03304.

[18] Dietmar Jannach. "Evaluating conversational recommender systems: A landscape of research". In: Artif. Intell. Rev. 56.3 (July 2022), 2365–2400. ISSN: 0269-2821. DOI: 10.1007/s10462-022-10229-x. **available at**: https://doi.org/10.1007/s10462-022-10229-x.

[19] Pablo Mateos and Alejandro Bellogín. "A systematic literature review of recent advances on context-aware recommender systems". In: Artificial Intelligence Review 58.1 (2024), p. 20. DOI: 10.1007/s10462-024-10939-4. **available at**: https://doi.org/10.1007/s10462-024-10939-4.

[20] Rishabh Mehrotra et al. "Towards a Fair Marketplace: Counterfactual Evaluation of the trade-off between Relevance, Fairness & Satisfaction in Recommendation Systems". In: Oct. 2018, pp. 2243–2251. DOI: 10.1145/3269206.3272027.

[21] Filip Radlinski and Nick Craswell. "A Theoretical Framework for Conversational Search". In: CHIIR 2017. ACM, 2017. **available at**: https://www.microsoft.com/en-us/research/publication/theoretical-framework-conversational-search/.

[22] Ruixuan Sun et al. Large Language Models as Conversational Movie Recommenders: A User Study. 2024. arXiv: 2404.19093 [cs.IR]. **available at**: https://arxiv.org/abs/2404.19093.

[23] Kerui Xu et al. "A Tag-Based Post-Hoc Framework for Explainable Conversational Recommendation". In: Proceedings of the 2022 ACM SIGIR International Conference on Theory of Information Retrie ICTIR '22. Madrid, Spain: Association for Computing Machinery, 2022, 232–242. ISBN: 9781450394123. DOI: 10.1145/3539813.3545120. **available at**: https://doi.org/10.1145/3539813.3545120.

[24] Yaochen Zhu et al. Collaborative Retrieval for Large Language Model-based Conversational Recommender Syste 2025. arXiv: 2502.14137 [cs.IR]. **available at**: https://arxiv.org/abs/2502.14137.